

Applications Development Practice 3 (ADP372S)

Assignment 1

Due Date: Sunday, 03rd April 2022

Lecturers: Arinze Anikwue, Radford Burger, Kruben Naidoo

Section A: Overview

The aim of the assessment will be to ensure that the learner has installed the appropriate IDE on his/her pc and demonstrate the ability to use **Git** and **Github** to collaborate on the same project with other members of the team. Furthermore, the learners will be assessed on the implementation of the **Maven** build in their project, and also show how **Test Driven Development** (TDD) is applied in the project solution.

The group will articulate their domain problem as a UML diagram. Thereafter each team member will implement a portion of the domain using the concepts of Domain Driven Design described thus far.

The following sections discuss the requirements in more detail.

Section B: Setting up the project and collaborating on Github

1. Each team member will setup/create your GIT account on **Github.com**.
2. The team leader will then:
 - a. create a repository on **Github.com** and;
 - b. set up a **maven project** in Java using **IntelliJ** and upload to your **Github.com** repository and;
 - c. invite team members as collaborators.
3. Each of the team members will then accept the collaboration invitation, and **fork** the project onto their **Github.com** account.
4. Each member (including the team leader) should create a **branch** (use student number as the branch name) and include a small program in the branch that demonstrate the **(TDD)** test fixtures as appropriate.
5. After the above-mentioned program code is completed on your branch, merge from your branch into the master, and send a pull request from your master to the team leader.
6. Thereafter the team leader will add the member's contributions to the master branch.
7. Each member should/must get the updated version of the project before submission.



Hint: Kindly refer to the notes and video recordings posted on Blackboard.

Section C: UML

The group would choose an appropriate problem domain to model. The diagram should depict all the Domains/Entities and their relationships. This problem should **NOT** be the same problem as the one you are working on in Project 3 (PRT362S). You may use StarUML (or any other modelling tool) to construct your diagram. Each team member will then implement at least ONE entity from the UML diagram as part of their program solution. More details are discussed below.

Section D: Implementing the UML domain problem using IntelliJ

In the real world, it is known that the success of a project is usually the result of what is known as careful planning, which goes hand in hand with the talent, and collaboration of a project's team members. This goes further with the formation of team and group projects. Consequently, when a company forms these teams, the members of the group may behave in whatever specific way, be working in individual parts. This is normal, however, it is in the best interest of the management to pay attention to **team roles** and **responsibilities**. **This is not done to micromanage the members, however,** to ensure the team works in a very effective manner. As a team, you have all come together and select an approved **design problem** in a **problem domain**. You have also designed a **domain model (UML class diagram)** for your design problem depicting **your domains (entities) and their relationships**.

Section D1: DDD- Domain

The first coding part of the practical assignment is based on the fact that:

You need to write code for your entities in the domain model using Builder Pattern. Entity is one of the building blocks of Domain Driven Design.

Activities

1. Pre-coding Design Activities

The group must design their Domain model using StarUML or any tool of your choice.

2. Group Lead Tasks:

- Create a maven application using IntelliJ that will act as the code base for your project.
- Create **one entity** as shown on your domain model using the **Builder Pattern**. Remember entities must be in a package called **Entity**.
- You also need to **create a repository** on Github for your project and **push the code base** to it.
- Add your group members as collaborators on the GITHUB repository and make sure they accept.
- Suggestion: Create a Milestone with a due date: **Monday, 14th March 2022**, and title: **Entity Milestone**.
- Create issues (creation of entities using Builder Pattern) and assign them to all group members, also assign the issues to the Entity Milestone. Each group member must be responsible for at least one entity.
- Merge correct, error-free pull requests from group members.

3. Group Members Tasks:

- Accept the collaboration invitation sent by your group leader on Github.
- Fork the Github repository created by your **group lead**, and then make a **clone** of your **remote** on your **local** repo.
- Solve the issue (in this case, create the code for an entity using Builder Pattern) assigned to you by your Group Lead.
- Push your code to your upstream.
- Send your code (using pull request) to the Group Lead.
- As soon as the group lead has updated the repository, get the updated version.

Section D2: DDD- Factory

The second coding part of the practical assignment is the factory for your entities. **Factory** is another building block of Domain Driven Design.

Activities

1. Group Lead Tasks:

- Create a **factory** package under your root package.
- Create one factory class for an entity. Use TDD.
- Update your repository on GITHUB with the new code.
- Suggestion: Create a Milestone with a due date: **Monday, 21st March 2022**, and title: **Factory Milestone**.
- Create issues (creation of factory classes) and assign them to all group members, also assign the issues to the **Factory Milestone**. Each group member must be responsible for at least one entity.
- Merge correct, error-free pull requests from group members.

2. Group Members Tasks:

- Get the updated GITHUB repository from your group lead.
- Solve the issue assigned to you by your group lead. Use TDD.
- Send (using pull request) your code to the group lead.
- As soon as the group lead has updated the repository, get the updated version.

Section D3: DDD- Repository

Your next practical coding task is the repository. Repository is another building block of Domain Driven Design.

Activities

1. Group Lead Tasks:

- Create a **repository** package under your root package.
- Create the main generic repository interface called **IRepository** (with the create, read, update and delete methods) in the root of the repository package.
- Create a Repository interface with its implementation (**in the impl package**) for an entity. Use TDD.
- Update your repository on GITHUB with the new code.
- Suggestion: Create a Milestone with due date: **28th March 2022**, and title: **Repository Milestone**.
- Create issues (creation of repository interface and implementation classes) and assign them to all group members, also add the issues to the **Repository Milestone**. Each group member must be responsible for at least one entity.
- Merge correct, error-free pull requests from group members.

2. Group Members Tasks:

- Get the updated GITHUB repository from your group lead.
- Solve the issue assigned to you by your group lead. Use TDD.
- Make sure your repository interface and its corresponding implementation are in the right packages.
- Send (using pull request) your code to the group lead.
- As soon as the group lead has updated the repository, get the updated version.

All group members must submit their copy (origin) of the GITHUB repository. That is the repository on your GITHUB account, which must be in sync with your group lead's GITHUB repository.

Section E: Test Driven Development

Each team member will have test classes (in the appropriate Test Packages) for the Domain/Entity they implemented. These test fixtures will be for testing the Factory and Repository.

All group members must submit **their copy (origin) of the GITHUB repository**. **Note:** That is the repository on your **GITHUB account**, which must be in **sync** with your **Group Lead's GITHUB repository on/before Sunday, 03rd April 2022**.

Section F: Submission

- You should complete the assignment with the same members in your Project 3 group. These groups have been set up on Blackboard. Should you not be part of a group in Project 3 or have problems contacting your team members, then send an email to Mr A Anikwue (anikwuea@cput.ac.za) as soon as possible.
- The **Author** (group member name + student number) can be included in the comments of each Java source file in the **IntelliJ** project where applicable. More information to follow in class.
- This is a coding project and as such the members of the group must demonstrate that they have contributed equally to the coding of the application and that the work was distributed fairly. *Marks may be awarded differently for each member based on their contributions.*
- Please note that all students are required to submit the **link to your GitHub repository ONLY** and not the project solution on Blackboard.
- **The due date for this assignment is Sunday 03rd April 2022 at 23h59.**

Further note: all members should add appropriate comments to their program code for example the following sample comments should be placed at the top of your source code file,

```
/* Employee.java
   Entity for the Employee
   Author: Mandisa Elizabeth Msongelwa (217149073)
   Date: 24 May 2021
  */
```